

Scenario Generation

I. METHODOLOGY

A. Scenario Generation

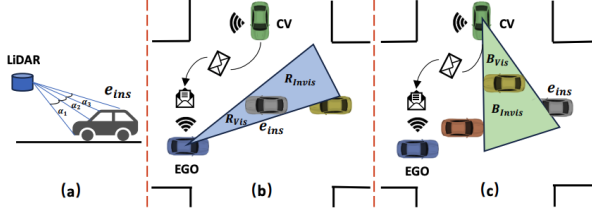


Fig. 1: Virtual LiDAR equipped by the V2X cooperative perception system emitting lasers to hit the inserted $entity_{e_{ins}}$. (a) View along the y-axis, (b) and (c) top view.

1) *Scenario Generation*: After computing the position information, we use MultiTest [1] for scene rendering, as follows: it invokes a simulated LiDAR to render the entity vehicle at the specified position, generating point clouds. We must remove all occluded point clouds. Finally, we generate point clouds and labels for all cooperative vehicles and the ego vehicle, and save the results.

The V2X communication system determines the pose transformation matrix among multiple agents to ensure that perception data measured in different local coordinate systems can be mutually converted. We therefore establish a unified world coordinate system to represent this transformation. The transformation from the world coordinate system to vehicle V_x ($x \in \{ego, cv_i\}$, where $i \in \{1, 2, \dots, n\}$) is expressed as:

$$\mathbf{p}_{V_x}^t = \mathbf{T}_{world \rightarrow V_x}^t \cdot \mathbf{p}_{world} \quad (1)$$

where $\mathbf{p}_{world} = [\omega x_0, \omega y_0, \omega z_0, 1]^T$ denotes a point in the world coordinate system, $\mathbf{T}_{world \rightarrow V_x}^t \in \mathbb{R}^{3 \times 4}$ is the coordinate transformation matrix from the world coordinate system to the local coordinate system of participant V_x at time t , and $\mathbf{p}_{V_x}^t$ is the position of participant vehicle V_x in the cooperative perception system at time t . Based on Equation 1, we apply the transformation matrix to convert world coordinates into the local frame of any participant vehicle. Conversely, each participant vehicle uses the corresponding inverse matrix to transform local coordinates back to the world coordinate system. This transformation mechanism ensures that subsequently inserted vehicles strictly maintain multi-view consistency.

In V2X-ATCT, vehicle insertion generates plausible new point cloud scenes s' by adding entities into the original multi-view scene $s = \{P_{ego}, P_{cv_i} \mid i \in \{1, 2, \dots, n\}\}$. Given an entity instance e_{ins} selected from the entity asset library E , we first compute valid locations and orientations for the entity in each view. We then render the entity instance as object-level point clouds using a set of virtual LiDARs and integrate

them correctly into the scene at corresponding viewpoints by modeling their interactions with background objects. We detail each step below [2].

Step 1: Valid Location Calculation. Inserting entities at appropriate locations is crucial for synthesizing realistic test scenes. Each inserted entity must be placed reasonably (e.g., a vehicle should be correctly positioned on the road) and satisfy the transformation constraints across multiple views. To compute feasible candidate insertion points, V2X-ATCT first employs a semantic segmentation model [3] to extract road point clouds from the ego vehicle's point cloud P_{ego} and each cooperative vehicle's point cloud P_{cv_i} , respectively. We then compute candidate insertion points in the world coordinate system and filter out any point that does not lie on the road surface in any view based on Equation 1, ensuring all candidate locations are valid from every viewpoint.

Additionally, inserting entity e_{ins} with specific orientation and dimensions at a candidate location may result in collisions with existing objects in the background scene s . To address this, V2X-ATCT computes the 3D bounding box B_e for the entity to be inserted and checks for overlap with all background object bounding boxes in each view. If the IoU between B_e and every background bounding box is zero, we consider the location collision-free.

Moreover, to ensure that at least one cooperative vehicle can detect the inserted entity, V2X-ATCT calculates the occlusion rate of the entity from multiple viewpoints and ensures that the occlusion rate from at least one cooperative vehicle's perspective remains below 90%.

Step 2: LiDAR Simulation and Insertion. Given a candidate insertion location $loc = \{loc_{ego}, loc_{cv_i} \mid i \in \{1, 2, \dots, n\}\}$, this step renders the entity instance into the original scene $s = \{P_{ego}, P_{cv_i} \mid i \in \{1, 2, \dots, n\}\}$ to generate a new, perspective-consistent scene s' . The realism of the synthesized scene depends on two key factors: maintaining perspective consistency across views and ensuring physical plausibility in each individual view. To achieve this, we adopt a high-fidelity approach to simulate the operational mechanism of cooperative perception LiDARs in the physical world.

Specifically, we construct a multi-view virtual LiDAR sensor set for V2X scene generation. Each participant is equipped with a configurable virtual LiDAR, initialized according to real-world LiDAR configurations in the V2X system (e.g., field of view, number of beams). The virtual LiDAR employs a ray-casting algorithm to simulate real laser beam emissions, rendering the inserted entity e_{ins} as object-level point clouds consistent with the respective viewpoint. Figure 1(a) illustrates the ray-casting process of a nonlinear LiDAR in a V2X system.

We further model interactions between the inserted entity

and background objects at each viewpoint. For each view, we remove background points occluded by the inserted entity and eliminate the occluded portions of the inserted entity based on geometric relationships. Figure 1(b) and Figure 1(c) demonstrate this processing in different views. In Figure 1(b), rays from the ego vehicle’s virtual scanner intersect the inserted entity e_{ins} , forming a visible region R_{vis} and an invisible region R_{invis} ; V2X-ATCT removes points in R_{invis} . In Figure 1(c), from the cooperative vehicle’s perspective, the inserted entity e_{ins} is partially occluded by a yellow background object; V2X-ATCT then removes the occluded parts in B_{invis} .

For each frame of point cloud data in the scene, we update the corresponding annotations. Specifically, both the ego vehicle and all cooperative vehicles (cv_i) possess associated point clouds and labels. The labels typically include fundamental information about the vehicle (e.g., speed, position), information on all detected traffic participants (e.g., positions, 3D bounding boxes), and other auxiliary data (e.g., pose transformation matrices). After inserting a vehicle into a given frame, we determine which cooperative vehicles can detect the inserted vehicle. If the inserted vehicle falls within the detection range of a cooperative vehicle, we update that vehicle’s label with the relevant information of the inserted entity. V2X-ATCT performs this label updating process iteratively until all frames in the scene have been fully processed.

REFERENCES

- [1] X. Gao, Z. Wang, Y. Feng, L. Ma, Z. Chen, and B. Xu, “Multitest: Physical-aware object insertion for testing multi-sensor fusion perception systems,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [2] A. Guo, X. Gao, C. Fang, H. Tian, W. Sun, Y. Mu, S. Tang, L. Ma, and Z. Chen, “Generate realistic test scenes for v2x communication systems,” *arXiv preprint arXiv:2506.07419*, 2025.
- [3] T. Cortinhal, G. Tzelepis, and E. E. Aksoy, “Salsanext: Fast, uncertainty-aware semantic segmentation of lidar point clouds for autonomous driving,” 2024. [Online]. Available: <https://arxiv.org/abs/2003.03653>